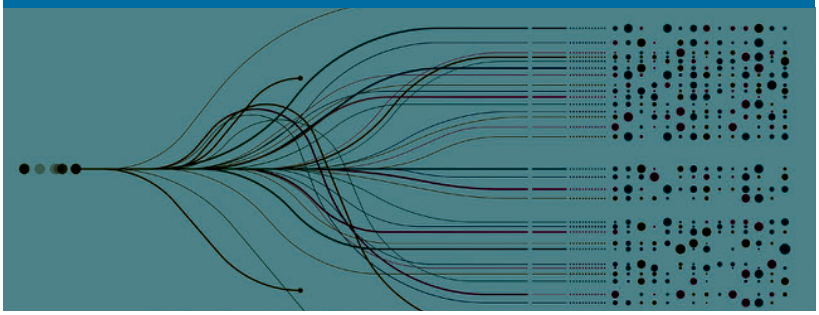
Large Language Models as Assistants in Software Architecture Design

Humberto Cervantes, Universidad Autonoma Metropolitana Iztapalapa

Yuanfang Cai, Drexel University

Rick Kazman, University of Hawaii, Honolulu

// To achieve desired software quality attributes, architects must evaluate and choose from a myriad of design concepts, patterns, and technologies. This article explores how large language models can assist in this decision-making process. //



©SHUTTERSTOCK.COM/KROT_STUDIO

Digital Object Identifier 10.1109/MS.2026.3663353
Date of publication 12 February 2026; date of current version 30 June 2026.

0740-7459 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

SOFTWARE ARCHITECTURE DESIGN is a complex process that requires architects to make various decisions to satisfy multiple architectural drivers, including quality attributes, primary functional requirements, constraints, and other concerns. Architectural design typically involves identifying *design concepts*—proven solutions to common design problems (such as design patterns, tactics, or externally developed components, such as frameworks)—and adapting them to the specific design problem that is being addressed. These activities are called "*Design concept selection and instantiation*" in the attribute-driven design[1] process. Design concept selection is inherently complex: it requires substantial knowledge and experience to identify viable alternatives for a given problem. Once these alternatives have been identified, evaluating and choosing the best solution is equally challenging.

Identifying viable alternatives can be especially challenging for inexperienced designers, creating a barrier to entry for those new to the design process. There have been research proposals, such as pattern recommenders to address this problem, but their results and adoption have been limited.[2]

Despite recent advances in large language models (LLMs) that enable various AI-assisted software-related tasks, their application in architectural design remains largely unexplored.[3] The emergence of generative AI and LLMs, trained on extensive design literature, pattern catalogs, and other solutions, has opened new possibilities to address the design concept selection problem, by enabling users to rapidly identify, compare, and explore design alternatives, and also by facilitating choice

through rapid assessment of each option's pros, cons, and tradeoffs.

We evaluated LLMs utility in design-concept selection by first prompting an LLM to generate design alternatives for scenarios with specific quality-attribute requirements. We then asked multiple LLMs to choose among the proposed options and examined their consistency. Finally, we collected selections from human architects and compared them against the LLMs' choices.

Our results show that LLMs make largely consistent choices, but human architects' choices are far more diverse, often disagreeing with or distrusting LLM suggestions due to the models' limited contextual awareness and tradeoff analysis. Humans still need to be the ones making architectural decisions, but LLMs can help by generating design options, acting as a sounding board, stimulating ideas, and fostering discussion.

> Design concept selection is inherently complex: it requires substantial knowledge and experience to identify viable alternatives for a given problem.

Background

Design Concepts

Software architecture design involves decomposing a problem into smaller, easier to address design problems. These problems often recur and, in many cases, solutions to these problems have been made publicly available. Conceptual solutions include design patterns and architectural tactics. Concrete solutions include externally developed components, such as libraries or specific technologies. We refer to these recurring design solutions collectively as *design concepts*.[1]

The identification and selection of design concepts is a complicated task, especially given the existence of many competing alternatives. This is particularly true in the case of design patterns where a vast number of catalogs has been published over the years. Technologies also present a great number of alternatives. Consider databases, for instance: relational and nonrelational. Within the nonrelational family, variants include key-value, document, and columnar. Once a particular type of nonrelational database has been selected, there are still decisions to be made regarding the specific technology to be chosen (e.g., MongoDB) and its configuration.

Related Work

The challenge of selecting optimal design concepts has prompted a diverse body of research.

In their systematic literature review about software design pattern selection approaches, Naghdipour et al.[2] provide a comprehensive overview of existing methods from 2007 to 2021. Because of this time range, LLMs are not mentioned. The papers they reviewed follow a number of different approaches, including case-based reasoning, text classification, question-answer, anti-patterns, ontology, information retrieval, or combined approaches. The authors conclude that the field of design pattern selection is not yet fully mature and faces major issues to overcome. Key limitations and open issues identified in the sources include the use of limited data sets, the use of a limited number of design patterns (Gang of Four mostly[4]), and that the approaches are semiautomatic, which makes them time-consuming.

Regarding other types of design concepts, the systematic literature review from Nabot[5] examines the proposed approaches to selecting commercial off-the-shelf. The papers reviewed range from 1995 to 2023, and LLMs are also not present in the reviewed papers. The main techniques identified include multicriteria decision making, optimization methods, fuzzy logic and fuzzy sets, clustering-based methods, specific methods, and combined approaches. This study concludes that, despite extensive academic exploration, ad-hoc selection processes remain common in practice, indicating a gap between research and industrial adoption.

More recent approaches leveraging LLMs are still sparse. Supekar and Khande[6] propose an AI-driven recommendation system that provides real-time, contextually relevant suggestions for appropriate design patterns inside an integrated development environment. Diaz-Pace et al.[7] introduce ArchMind, an LLM-based

design assistant aimed at helping novice architects. ArchMind utilizes LLMs with retrieval-augmented generation (RAG) and prompting techniques. Dhar et al.[8] presents an exploratory empirical study investigating the feasibility of using LLMs to generate architectural design decisions. The key findings indicate that LLMs can successfully generate relevant and accurate design decisions, though they do not yet reach human-level performance. Sliman and Keim[9] present an exploratory case study evaluating the capability of LLMs to answer questions about the architectural knowledge of existing software systems. The authors conclude that LLMs do indeed contain software architectural knowledge. While our work is close to that of Supekar and Khande[6] and Diaz-Pace et al.,[7] our approach is different in that it does not specifically focus on design patterns only, and it leverages the fact that LLMs do contain architectural knowledge to propose other types of design concepts as possible solutions to design problems.

Research Questions

We hypothesize that LLMs can assist in software architecture design by generating and selecting from existing proven solutions, but they cannot replace human judgment in making context-aware decisions. Based on this hypothesis, we investigate three research questions, the first two of which answer the question of whether LLMs can be reliable design partners, and the third explores the degree to which LLMs and humans need to collaborate:

- *RQ1*—Can an LLM identify appropriate design concept alternatives for a given design problem?
- *RQ2*—Is there a consensus between different LLMs on the selection of an alternative for a design problem?
- *RQ3*—What is the relationship between the design decisions made by humans and LLMs given the same problem description?

Methodology

To answer the research questions we followed a methodology consisting of five steps:

1) Define a prompt to generate self-contained, well-defined design problems and design concept alternatives.
2) Generate and review a collection of design problems and design concept alternatives.
3) Ask different LLMs to select the most appropriate design concept alternative and study if there is consensus.
4) Ask humans to select the most appropriate design concept alternative for a number of design problems.
5) Compare LLM decisions with those of humans.

A replication package with artifacts associated with these steps is provided online.[a]

Prompt Definition

The first step of our methodology is to define a prompt to generate self-contained, well-defined design problems and design concept alternatives. Since we will ask the same design problems of humans, we favor short descriptions with the least possible ambiguity. The prompt we used is the following:

> You are an expert software architect, please be formal and professional in your responses . You want to create a software architecture design exam. This exam consists of design questions where the respondents must select a design pattern or technology to solve a particular design problem . I want you to make the different choices not so obvious, the exam taker needs to reflect very well to select an appropriate option. In other words, make it difficult.
>
> Questions should have:
>
> - Problem context : The context, while short, should be very clear and any term such as" mission - critical " or " high - volume " must be given a meaning and explained shortly (if possible, a metric should be given).
>
> - Options : Four different options should be presented in a table that includes name, short description, pros and cons . A suggested answer should also be included .

We selected four alternatives because we consider this to be a suitable number for human evaluation. As part of question generation, the prompt also requests the LLM to generate a candidate solution. We used a single LLM to generate the questions to maintain consistency and we selected ChatGPT 4o as ChatGPT was the most popular AI chatbot at the time of the study (March 2025).

Problem Generation

We created an initial set of 14 questions covering the most common quality attributes:

- availability
- security
- modifiability
- testability
- scalability
- integrability
- usability
- cost-efficiency
- deployability
- observability
- eventual consistency
- transactional consistency

[a] https://tinyurl.com/replication-pkg

An example of the questions that are generated is the following, with specific availability requirements:

You are designing a payment processing system for a rapidly growing e - commerce platform. The system must process 10,000 transactions per second (TPS) with a target availability of 99.999% (5 nines), meaning the system can only be down for 5.26 min per year. The system handles critical financial transactions where consistency is paramount, and any data inconsistency could lead to financial losses or regulatory issues.

The current system processes payments in a single region and has experienced several outages due to natural disasters and network issues . The business has decided to implement a multi - region strategy to improve availability .

Which approach best addresses the availability requirements of this system ?

LLM Design Concept Selection

We then asked four different LLMs to provide answers to the design questions. The LLMs chosen were:

- ChatGPT 4o
- DeepSeek R1
- Claude 3.7
- Gemini 2.5 Pro

These LLMs were chosen because they were among the most widely used at the time of the study,[b] and they provide a web-based conversational UI. We opted for this approach as this is the most common way users interact with LLMs. Also, we selected the most recent versions available at the time of experimentation.

Human Design Concept Selection

During the Designing 2025 workshop, we conducted an interactive session designed to explore how software practitioners reason about architectural decisions and how their reasoning compares with leading AI models. The 23 participants were recruited from the workshop attendees and included six industry professionals (designers, software architects, and developers), 11 academic researchers specializing in software architecture and education, and six graduate students in the field of software architecture. Nearly 75% had more than five years of software development experience, and over 60% had equivalent experience in software architecture and design. This indicates that the discussion was shaped by a well-informed group with substantial practical and theoretical expertise in software design.

The participants were presented with five out of the original 14 application scenarios, each representing an architecture challenge. The questions that were selected were 1 (availability), 2 (security), 5 (transactional consistency), 7 (testability), and 8 (scalability).

Each participant accessed the scenarios by scanning a QR code, then independently answered five decision-making questions (requiring approximately 5–6 min each, for a total of 20–30 min). These questions were designed to elicit explicit tradeoffs, underlying assumptions, and context-sensitive rationale behind architecture choices.

After completing their individual responses, participants were shown answers generated by the four state-of-the-art AI models. Participants then self-organized into four groups. Each group engaged in a 20–30 min discussion to compare human and AI responses, reflect on divergent rationales, and evaluate the quality and completeness of the AI-generated decisions. Each team was asked to reflect on and respond to the following discussion questions (DQ1–DQ4), addressing implications across practice, education, and research:

- *DQ1*: *Overall*—Do you agree with the AI-generated decisions?
- *DQ2*: *Industry*—To what extent do you trust AI to make architectural decisions in an industrial setting, especially when it provides no reasoning?
- *DQ3*: *Education*—How should software design and architecture curricula evolve to integrate AI effectively?
- *DQ4*: *Research*—What should researchers focus on or engage in to enhance AI's ability to support software design?

A QR code was provided to each group for submitting their responses, enabling real time capture of insights. The session concluded with a summary phase (20–30 min), during which each group synthesized their main discussion points in a shared Google Slides deck. A representative from each group then presented their findings to all participants, fostering cross-group reflection and further dialogue. It should be noted that the study involved an anonymous survey with no collection of personal or identifying information, so this does not constitute human subject research. Additionally, the participants were informed that we planned to publish the results.

Results

RQ1: Identification of Alternatives

As mentioned previously, we asked the LLMs to generate four alternatives for each design problem. For the availability question presented previously, the alternatives that were proposed include:

[b]According to https://lmarena.ai/leaderboard

- active–passive replication
- active–active with conflict resolution
- event sourcing with command query responsibility segregation (CQRS)
- sharded database with cross-region replication.

For each of the alternatives, a description and a list of pros and cons was also generated. For example, for active–active replication, the description is:

- Both regions actively process transactions with eventual consistency. Conflicts are resolved using timestamps and business rules.

The list of pros includes:

- true multiregion availability
- lower latency for all operations
- better resource utilization
- no single point of failure.

And the list of cons includes:

- complex conflict resolution logic
- risk of temporary inconsistencies
- higher development and testing effort
- more complex monitoring and debugging.

The authors reviewed the options proposed by the LLMs for each question. In each design problem, the alternatives that were proposed were aligned with the problem presented. Across the 14 questions and four alternatives to each one of them (56 alternatives), none was flagged as being inappropriate as validated by the authors. In addition, for the five scenarios and alternatives presented to the workshop participants, none of them were flagged as inappropriate. We thus conclude that the alternatives identified by the LLM are appropriate and relevant, even across a diverse set of design problems.

The complete list of questions with their alternatives can be found online.[c]

Table 1. Design concept selection across LLMs.

#	QA	LLMs in agreement	Most common selection
1	Availability	2 (Claude, GPT 4o)	Active–active with conflict resolution
2	Security	2 (Claude, Gemini)	Zero trust architecture with identity-aware proxy
3	Modifiability	4	Microservices with internal API contracts
4	Eventual consistency	5	Reservation system with compensating transactions
5	Transactional consistency	4	SAGA pattern (orchestration)
6	Availability	5	Event-based asynchronous communication via message broker
7	Testability	5	Hexagonal architecture with in-memory adapters
8	Scalability	5	Command query responsibility segregation (CQRS)
9	Security	4	Service mesh with mTLS and identity-based policies
10	Integrability	4	API gateway + adapter services
11	Usability	5	Headless BI with semantic layer
12	Cost-efficiency	5	Pre-rendered static content on CDN + on-demand APIs
13	Deployability	5	Microservices with CI/CD and canary releases
14	Observability	5	OpenTelemetry-based distributed tracing

API: application programming interface; mTLS: mutual transport layer security headless; BI: headless business intelligence; CDN: content delivery network; CI/CD: continuous integration/continuous deployment.

RQ2: Consensus Among LLMs in the Selection of Design Concepts

Table 1 summarizes the results of our analysis regarding whether or not there is consensus between LLMs in the selection of design concepts.

In questions 1 (availability) and 2 (security), the LLMs exhibit notable disagreement (2 out of 5 agreed). For example, in the availability scenario, Claude 3.7 and ChatGPT 4o favored active–active with conflict resolution, while DeepSeek and Gemini 2.5 Pro recommended sharded database with cross-region replication. Interestingly, ChatGPT provided two different answers when asked the same question twice (active–active and

[c]https://tinyurl.com/llm-questions

active–passive). A similar divergence was observed in the security scenario.

Even though there was disagreement in two of the 14 questions, we observe that the LLMs were generally in agreement (at least four out of five agreed) with respect to which design concept is better suited to address the given design problems. The complete list of answers to the questions by the different LLMs can be found online.[d]

RQ3: Human and LLM Design Decisions

Figure 1 presents a comparative analysis of architectural decisions made by the LLMs as well as by 23 human participants across five scenarios corresponding to questions 1,2,5,7, and 8 in Table 1. A summary of the results from each group can be found online.[e]

In contrast to the LLMs' behavior, the human participants consistently demonstrated a much more diverse decision-making pattern across all five scenarios. Their selections were more widely distributed among the available options, and a substantial portion of them indicated that more information was needed. This highlights the caution exercised by human architects when making decisions under uncertainty——something LLMs do not currently emulate well.

In the last two scenarios——testability and scalability (questions 7 and 8)——all LLMs converged on a single architectural recommendation. While this uniformity might appear to indicate confidence or clarity, it may instead reflect potential overfitting or biases in the models' training data. In contrast, the human participants provided varied responses and showed more willingness to consider multiple approaches.

In particular, for the security scenario, LLMs align more closely with human experts: both selected the same top three choices. For both the availability and security scenarios where the four LLMs disagreed, these divergent answers were also chosen by different human experts. Notably, in the scenarios where all LLMs unanimously made the same decision, human decisions showed the greatest divergence. A notable case is testability, where all LLMs agreed on a hexagonal architecture with in-memory adapters, but no humans selected this option. A possible explanation may be that this architecture, which is indeed well suited for testability, was not so well known by many people, as opposed to microservices which were favored by the participants.

To summarize our research findings: the LLMs were able to identify appropriate design concept alternatives (RQ1) and multiple LLMs achieved significant consensus (RQ2) but these decisions were not well aligned with those of humans (RQ3). In particular, the decisions made by the workshop participants were significantly different than those of the LLMs, as shown in Figure 1. These findings collectively raise important questions about the maturity of LLMs in supporting architectural decision-making and reinforce the continuing importance of human oversight.

Workshop Summary

The majority of participants were experienced professionals and academics. The responses and presentations of the four groups offer valuable insights into the current perceptions and expectations of AI in software design across academia and industry. We now present their consolidated responses with respect to each discussion question.

DQ1: Overall

Across all groups, participants expressed skepticism regarding the AI-generated architectural decisions. **Group 1** emphasized that different AI models often produce inconsistent results, raising concerns about reliability. They noted that architectural decisions involve tradeoffs across multiple quality attributes (e.g., availability, testability), which require contextual and stakeholder-specific considerations often missing from AI outputs. **Group 3** echoed that there should be zero-trust to AI due to "the lack of transparency/motivation." **Group 2** highlighted that agreeing or disagreeing with AI recommendations depends on one's own confidence and expertise. **Group 4** echoed similar concerns, pointing out that without insight into the AI's reasoning, users remain uncertain, especially when LLMs are known to exhibit hallucinations.

DQ2: Industry

When asked whether AI should be trusted in making architectural decisions in industrial settings without providing reasoning, all four groups reported *low to zero trust*. **Group 2** noted that even if reasoning is provided, human oversight is still essential to evaluate its soundness. **Group 4** raised the risk that AI systems can produce plausible sounding justifications for nearly any decision, which may mask deeper issues and mislead teams.

DQ3: Education

Despite the aforementioned concerns, participants acknowledged AI's potential as a valuable educational tool. All groups agreed that AI should serve to support, not replace, human creativity and critical thinking. **Group 1** compared AI to a research assistant——helpful in

[d]https://tinyurl.com/llm-answers
[e]https://tinyurl.com/workshop-results

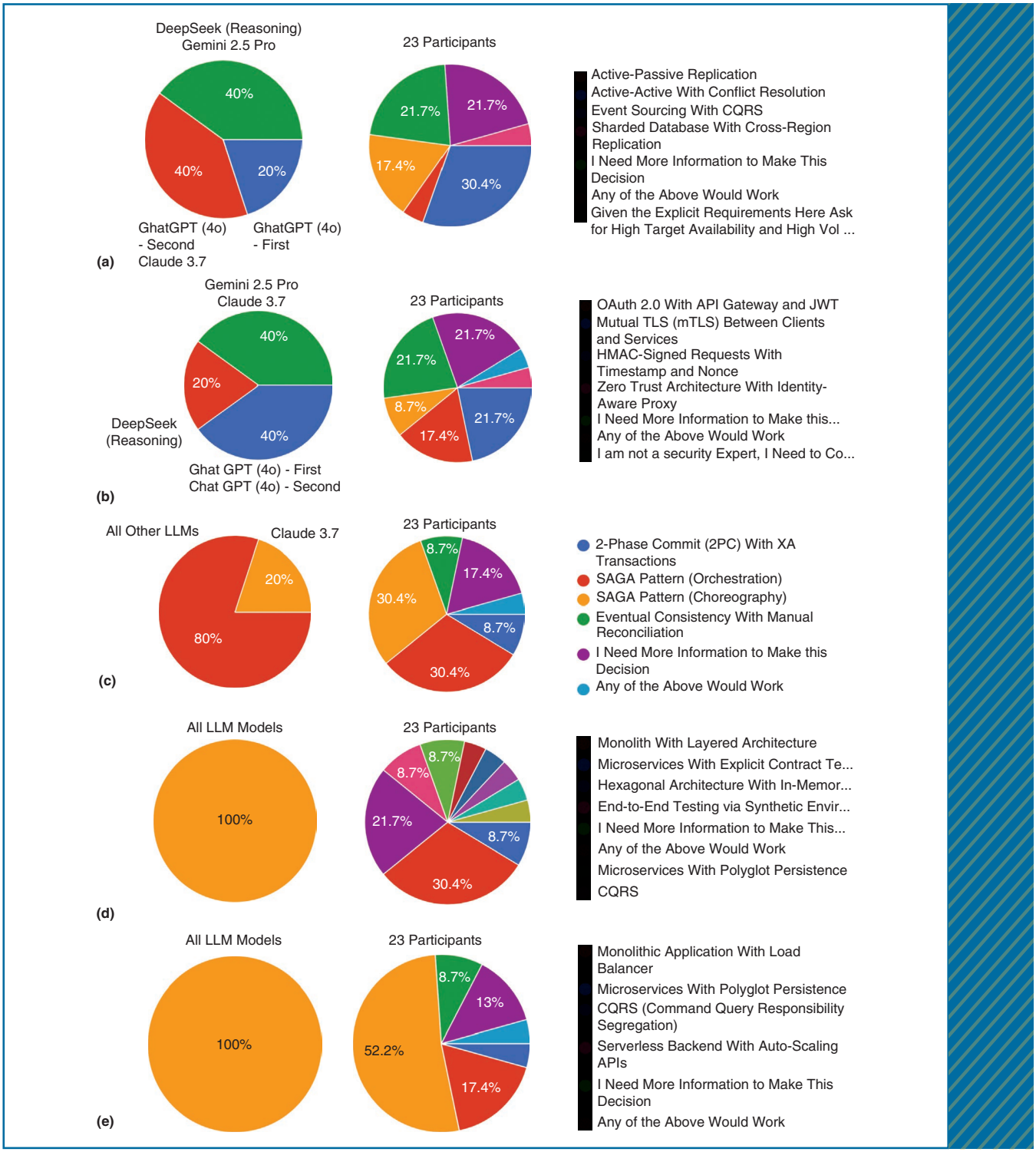


FIGURE 1. Human decisions versus AI decisions. (a) Availability. (b) Security. (c) Transaction consistency. (d) Testability. (e) Scalability. API: aplication programming interface; JWT: JSON web token; HMAC: hash-based message authentication code; XA: extended architecture.

brainstorming and prototyping but ultimately subordinate to human judgment. **Groups 2** and **4** suggested a teaching model in which students critique AI-generated designs to deepen understanding. **Group 3** proposed using AI to generate personas, stakeholder models, and design critiques, while adopting a critical attitude toward AI responses. Prompt engineering, they noted, should become a fundamental skill in software design education.

DQ4: Research

Participants identified several research priorities for improving AI's role in software architecture. **Transparency and accountability** emerged as the top concerns across groups. **Group 1** called for exploring models beyond LLMs and considering the carbon footprint of AI systems. **Group 2** emphasized the need to handle software-specific artifacts (e.g., unified modeling language diagrams) and to enable more robust reasoning and feedback loops. **Group 3** reiterated the need for explainability. **Group 4** added that quality attributes, such as maintainability, security, and ethical considerations should be central to future AI systems in this domain.

The inconsistencies noted underscore a fundamental limitation of current LLMs in reasoning about architectural tradeoffs, especially when contextual understanding and nuanced decision-making are required.

> LLMs do not have the breadth of context that is required to make many important design decisions.

Given the results from our workshop, we can conclude that, at least for the moment, no architect's job is in danger. LLMs do not have the breadth of context that is required to make many important design decisions. In the real world, an architect might consider cost, experience of the team, time to market, product familiarity, prototyping results, compatibility with other chosen technologies, and a myriad of other factors when making a design choice. While some of these factors might be brought into the prompt given to an LLM, we believe that it is safe to say that their limited scope of context and knowledge will prevent them from making decisions that a project team will blindly accept. Thus human input is crucial.

So how can an architect use LLMs profitably? Our experience in interacting with these LLMs to produce design alternatives, and our experience with the workshop, suggests an enticing middle ground. Design is a creative process. And the creative process involves a combination of divergent thinking—generating multiple, novel, and varied ideas or solutions to a problem—and convergent thinking—putting multiple diverse ideas together into a coherent whole. While LLMs may not excel at the latter, they can be extremely useful in the former.

LLMs can aid an architect in "scanning the horizon" for possible solutions and quickly enumerating their well-known pros and cons. This can make the search for appropriate design concepts much more thorough: the LLM may have access to far more relevant data about design concepts than your average architect, and more efficient: the LLM, by enumerating pros and cons, can quickly help an architect eliminate large numbers of alternatives. In this way the architect's time can be spent in detailed analysis and comparison of a set of design concepts that both cover the horizon of possibilities and have been vetted, at least in part.

We therefore believe that, at least for the moment, they can serve as a competent assistant to, and sounding board for, the architect.

Future Work

This article presented an initial exploration of using LLMs to assist in the selection of design concepts. Several avenues for future research remain open:

- *Exploring why decisions differ.* The workshop revealed only that human experts and AI models make decisions in markedly different ways. What remains unclear, and deserves further investigation, is why these differences arise. We need more in-depth study to uncover the factors that shape human judgment versus the reasoning process of LLMs, including aspects such as training data or configurations such as temperature.
- *More complex design problems.* The design problems used in this

study were isolated from each other, which rarely reflects real-world system design. Investigating how LLMs perform in realistic case studies involving interrelated design decisions, constraints (e.g., team expertise, budget), and trade-offs is necessary to assess their effectiveness in broader contexts.
- *Identifying alternatives using multiple LLMs*. In this study, only one LLM was used to generate design concept alternatives. Comparing the suggestions from multiple models could help determine which models are most effective at proposing relevant and diverse alternatives.
- *Integration into architecture education*. Teaching software architecture is challenging in part because students often lack the experience to identify appropriate design concepts. LLMs could help bridge this gap, but the key challenge is ensuring that students use them as learning aids rather than becoming overly dependent on them.

ACKNOWLEDGMENT

We wish to thank the organizers and participants of the Designing 2025 workshop.

References

1. H. Cervantes and R. Kazman, *Designing Software Architectures: A Practical Approach*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2024.
2. A. Naghdipour, S. M. H. Hasheminejad, and R. L. Barmaki, "Software design pattern selection approaches: A systematic literature review," *Softw.: Pract. Experience*, vol. 53, no. 4, pp. 1091–1122, 2023, doi: 10.1002/spe.3176.
3. X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Trans. Software Eng. Method.*, vol. 33, no. 8, pp. 1–79, 2024, doi: 10.1145/3695988.
4. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley, 1994.
5. A. Nabot, "Software component selection methods and techniques: A systematic review," *Indonesian J. Elect. Eng. Comput. Sci.*, vol. 33, no. 3, pp. 1802–1811, 2024, doi: 10.11591/ijeecs.v33.i3.pp1802-1811.
6. V. Supekar and R. Khande, "Improving software engineering practices: AI-driven adoption of design patterns," in *Proc. IEEE Int. Conf. Adv. Comput. Commun. Technol. (ICACCTech)*, 2024, pp. 768–774, doi: 10.1109/ICACCTech65084.2024.00128.
7. J. A. Díaz-Pace, A. Tommasel, and R. Capilla, "Helping novice architects to make quality design decisions using an LLM-based assistant," in *Proc. Eur. Conf. Softw. Archit. (ECSA)*, 2024, pp. 324–332.
8. R. Dhar, K. Vaidhyanathan, and V. Varma, "Can LLMs generate architectural design decisions? - An exploratory empirical study," in *Proc. IEEE 21st Int. Conf. Softw. Archit. (ICSA)*, 2024, pp. 79–89, doi: 10.1109/ICSA59870.2024.00016.
9. M. Soliman and J. Keim, "Do large language models contain software architectural knowledge?: An exploratory case study with GPT," in *Proc. IEEE 22nd Int. Conf. Softw. Archit. (ICSA)*, 2025, pp. 13–24, doi: 10.1109/ICSA65012.2025.00012.

ABOUT THE AUTHORS



HUMBERTO CERVANTES is a professor and researcher at the Universidad Autónoma Metropolitana Iztapalapa, 09340 Mexico City, Mexico. His research interests include software architecture, technical debt, and AI-assisted design. Cervantes received his Ph.D. in computer science from Université Grenoble Alpes. Contact him at hcm@xanum.uam.mx.



YUANFANG CAI is a professor of computer science at Drexel University, Philadelphia, PA 19104 USA. Her research interests include software architecture, design evolution, and technical debt management. Cai received her Ph.D. in computer science from the University of Virginia. She is a Member of IEEE. Contact her at yuanfang.cai@drexel.edu.



RICK KAZMAN is the Danny and Elsa Lui Distinguished Professor of Information Technology Management at the University of Hawaii at Honolulu, Honolulu, HI 96822 USA. His research interests include software architecture, design and analysis tools, and technical debt. Kazman received his Ph.D. in computer science from Carnegie Mellon University. He is a Fellow of IEEE and serves on the IEEE Computer Society Board of Governors. Contact him at kazman@hawaii.edu.